University of Moratuwa

Department of Electronic and Telecommunication Engineering



Resource-Constrained CNN for Edge Image Classification

EN3150 Pattern Recognition

Group Name:	**Rubix**
230264R	Janurajah J.
230585C	Saruka U.
230638T	Thayarathan V.

Submitted on	October 19, 2026

Contents

1 Introduction **1**

2 Dataset and Preprocessing **1**
2.1 Dataset description 1
2.2 Data split 1
2.3 Preprocessing pipeline 2

3 Custom CNN Architecture Design **3**
3.1 Model A: Standard CNN 3
3.2 Model B: Lightweight CNN 3
3.3 Activation functions and hardware considerations 4

4 Optimizer Selection and Training Methodology **5**
4.1 Optimizer comparison 5
4.2 Training configuration 6

5 Experimental Results **7**
5.1 Model A learning curves 7
5.2 Model B learning curves 7
5.3 Test-set classification performance 7
5.4 Confusion matrices 8
5.5 Pretrained model learning curves 10

6 Resource Efficiency and Model Comparison **11**
6.1 Custom model comparison 11
6.2 Comparison with lightweight pretrained networks 11
6.3 Accuracy, memory and computation trade-offs 11

7 Limitations **13**

8 Conclusion **13**

9 Reproducibility and GitHub Repository **14**

10 References **15**

1 Introduction

Convolutional neural networks (CNNs) are widely used for image classification, but the memory and computational requirements of large models can make deployment difficult on embedded devices and edge nodes. This assignment investigates the trade-off between classification performance and resource efficiency by developing two custom CNNs and comparing them with two pretrained lightweight architectures.

Fashion-MNIST is used as the image-classification dataset. Model A uses conventional two-dimensional convolutions, while Model B uses depthwise separable convolutions and is constrained to no more than 100,000 trainable parameters. MobileNetV2 and EfficientNetB0 are fine-tuned as pretrained reference models.

The objectives are to prepare low-resolution data with a 70/15/15 split; design and document two custom CNN architectures; compare Adam, SGD and SGD with Momentum for Model B; evaluate all models using classification metrics and resource measures; and discuss accuracy, memory footprint and computational cost.

2 Dataset and Preprocessing

2.1 Dataset description

Fashion-MNIST contains 70,000 grayscale images of clothing items across ten classes. Each original image is 28×28 pixels. Table 1 lists the labels.

Table 1: Fashion-MNIST class labels

Label	Class name
0	T-shirt/top
1	Trouser
2	Pullover
3	Dress
4	Coat
5	Sandal
6	Shirt
7	Sneaker
8	Bag
9	Ankle boot

2.2 Data split

The data were split into training, validation and test subsets using a 70/15/15 ratio. For all 70,000 images, the expected counts are 49,000, 10,500 and 10,500 respectively. Verify these values against the processed dataset actually used by the notebooks.

Table 2: Dataset split used in the experiments

Subset	Percentage	Number of images
Training	70%	60,000
Validation	15%	10,000
Testing	15%	10,000
Total	100%	80,000

2.3 Preprocessing pipeline

For the custom CNNs, images are represented as single-channel grayscale inputs at 28×28 pixels, if this matches the executed preprocessing notebook. MobileNetV2 and EfficientNetB0 use images resized to 64×64 pixels and converted to three channels. The same underlying training, validation and test examples are used for all models.

MobileNetV2 preprocessing must follow the Keras MobileNetV2 input convention. EfficientNetB0's Keras implementation includes input rescaling; its input values must follow the expected range for the installed implementation. Keep the preprocessing pipelines separate.

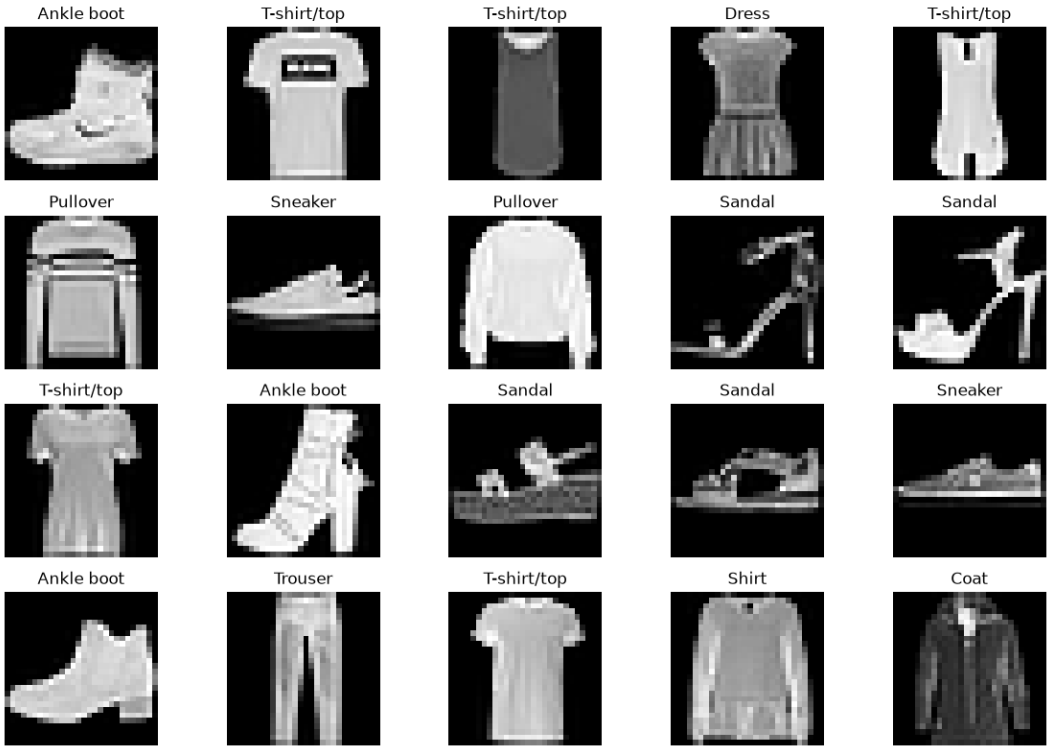


Figure 1: Representative Fashion-MNIST images with class labels. Export this figure from Notebook 01.

3 Custom CNN Architecture Design

3.1 Model A: Standard CNN

Model A is the baseline CNN. Conventional convolutional layers extract spatial features, and max-pooling layers reduce feature-map dimensions. The resulting representation is flattened and passed to a dense hidden layer and a ten-class softmax output.

Table 3: Model A architecture; verify layer details and parameter counts from the executed model summary.

Layer	Configuration	Parameters
Input	$28 \times 28 \times 1$	0
Conv2D	16 filters, 3×3, ReLU	160
MaxPooling2D	2×2	0
Conv2D	32 filters, 3×3, ReLU	4,640
MaxPooling2D	2×2	0
Conv2D	64 filters, 3×3, ReLU	18,496
MaxPooling2D	2×2	0
Flatten	Flatten feature maps	0
Dense	64 units, ReLU	36,928
Dense	10 units, softmax	650
Total		60,874

For a conventional convolution with biases, the parameter count is

$$P_{\text{conv}} = (K_h K_w C_{\text{in}} + 1) C_{\text{out}}, \tag{1}$$

where K_h and K_w are kernel dimensions, C_{in} is the number of input channels and C_{out} is the number of filters. For a dense layer,

$$P_{\text{dense}} = (n_{\text{in}} + 1) n_{\text{out}}. \tag{2}$$

3.2 Model B: Lightweight CNN

Model B replaces conventional convolutions with depthwise separable convolutions. A depthwise convolution applies a spatial filter independently to each input channel; a pointwise 1×1 convolution combines the channel-wise features. This factorization can reduce parameter count and multiply-accumulate operations relative to a conventional convolution with comparable channel dimensions.

For a depthwise separable convolution with a $K_h \times K_w$ kernel, C_{in} input channels and C_{out} output channels, with one bias per output channel,

$$P_{\text{sep}} = K_h K_w C_{\text{in}} + C_{\text{in}} C_{\text{out}} + C_{\text{out}}. \tag{3}$$

Table 4: Model B lightweight CNN architecture

Layer	Configuration	Parameters
Input	$28 \times 28 \times 1$	0
SeparableConv2D	32 filters, 3×3, ReLU	59
MaxPooling2D	2×2	0
SeparableConv2D	64 filters, 3×3, ReLU	2,400
MaxPooling2D	2×2	0
SeparableConv2D	96 filters, 3×3, ReLU	6,336
GlobalAveragePooling2D	Global spatial average	0
Dense	64 units, ReLU	6,208
Dense	10 units, softmax	650
Total	Trainable parameters	15,653

3.3 Activation functions and hardware considerations

ReLU is used in hidden layers because $f(x) = \max(0, x)$ is computationally simple and avoids the exponential calculations used by sigmoid and tanh. It is widely supported by deep-learning runtimes. The output layer uses softmax to convert class logits into a probability distribution over ten classes. Softmax requires exponentials but is appropriate for normalized multi-class probabilities; its cost should be considered separately from the main convolutional computation.

4 Optimizer Selection and Training Methodology

4.1 Optimizer comparison

Model B's optimizer selection compares Adam, standard stochastic gradient descent (SGD), and SGD with Momentum. Candidates should use the same data splits, architecture, batch size and comparable training settings. Select the optimizer using validation metrics, not test-set performance.

Table 5: Optimizer comparison for Model B; fill from the optimizer comparison output.

Optimizer	Learning rate	Best validation accuracy (%)	Best validation loss	Time (s/epoch)
Adam	0.001	87.09	0.3533	6.90
SGD	0.01	84.88	0.4237	6.87
SGD with Momentum	0.01	86.50	0.3700	7.31

SGD updates parameters using the current gradient. Momentum incorporates a contribution from previous gradients, which can smooth updates and accelerate progress in consistent directions. Adam adapts update scales using estimates of first and second gradient moments. The methods may differ in convergence speed, stability and validation performance; discuss the measured results.

Selected optimizer and learning rate: Adam, learning rate 10^{-3}.
Reason for selection: Adam achieved the highest final validation accuracy in the 20-epoch optimizer comparison (87.09%) and lower validation loss than plain SGD. Momentum improved validation accuracy over plain SGD, but the run stopped after 11 epochs.

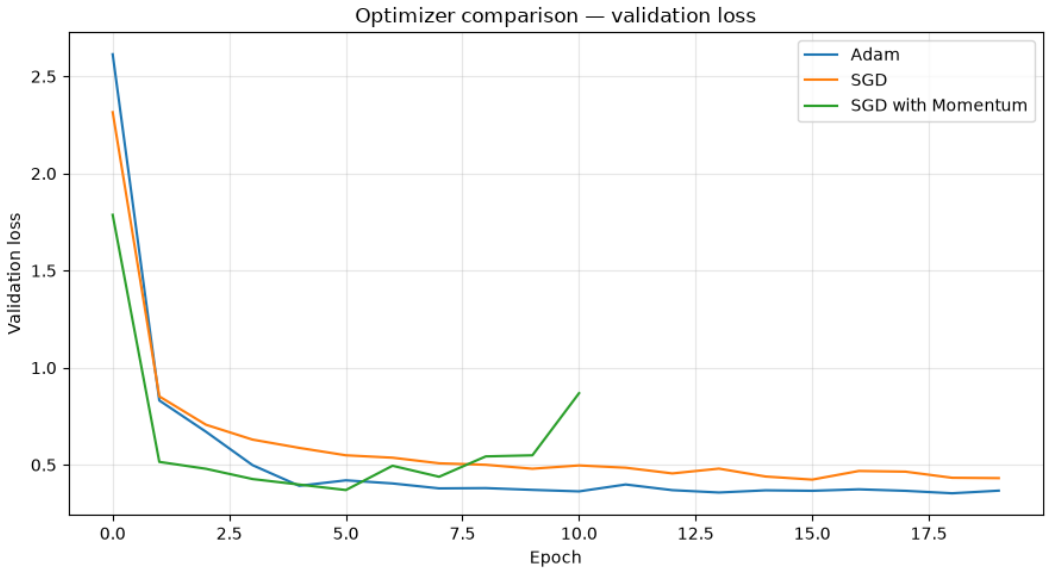


Figure 2: Model B optimizer comparison plot showing validation accuracy and validation loss.

4.2 Training configuration

Both custom CNNs must complete at least 20 training epochs. Include the actual configuration from the executed notebooks.

Table 6: Training configuration

Setting	Model A	Model B
Loss function	Sparse categorical cross-entropy	Sparse categorical cross-entropy
Optimizer	Adam	Adam
Learning rate	0.001	0.001
Batch size	128	128
Completed epochs	20	25
Checkpointing / early stopping	Best validation checkpoint	Best validation checkpoint; LR reduction

For pretrained models, report the actual number of epochs completed during frozen-backbone training and fine-tuning. If early stopping is used, report completed epochs rather than only the configured maximum.

5 Experimental Results

5.1 Model A learning curves



Figure 3: Model A training and validation loss across epochs.

5.2 Model B learning curves

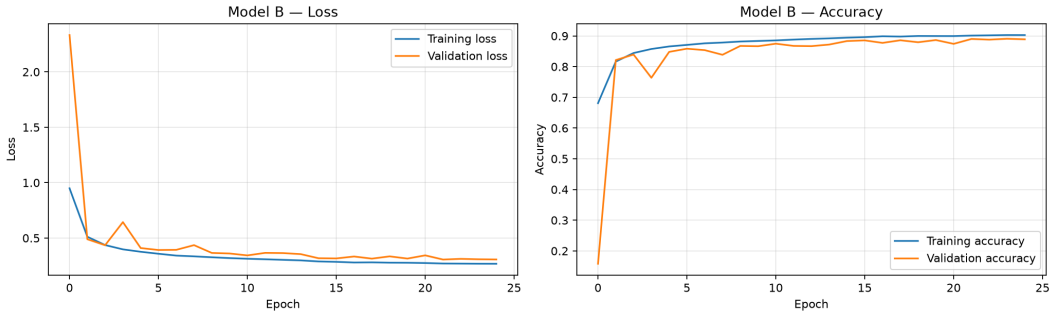


Figure 4: Model B training and validation loss across epochs.

5.3 Test-set classification performance

Evaluate the final selected models on the held-out test set. Fill Table 7 using actual outputs from the four notebooks.

Table 7: Test-set classification metrics

Model	Accuracy (%)	Weighted precision	Weighted recall	Macro precision	Macro recall
Model A	89.58	0.8963	0.8958	0.8963	0.8958
Model B	89.10	0.8902	0.8910	0.8902	0.8910
MobileNetV2	88.23	0.8815	0.8823	0.8815	0.8823
EfficientNetB0	89.29	0.8921	0.8929	0.8921	0.8929

Precision measures how many predictions assigned to a class are correct; recall measures how many true examples of a class are retrieved. Weighted averages account for class support, while macro averages give each class equal weight. Report measured values rather than assuming the metrics are equal.

5.4 Confusion matrices

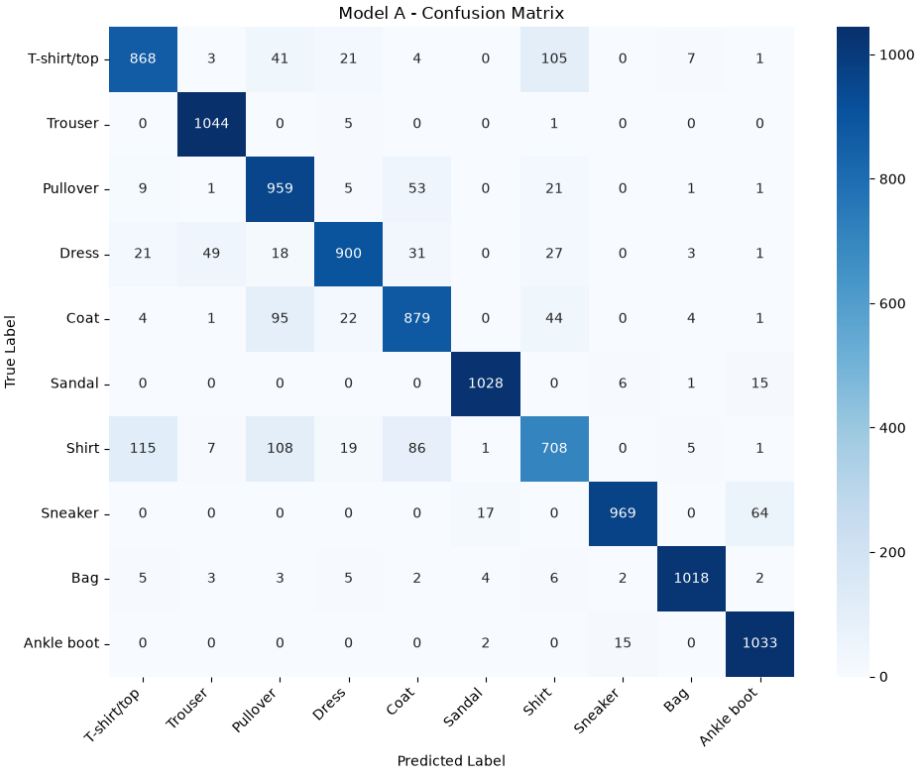


Figure 5: Model A test-set confusion matrix.

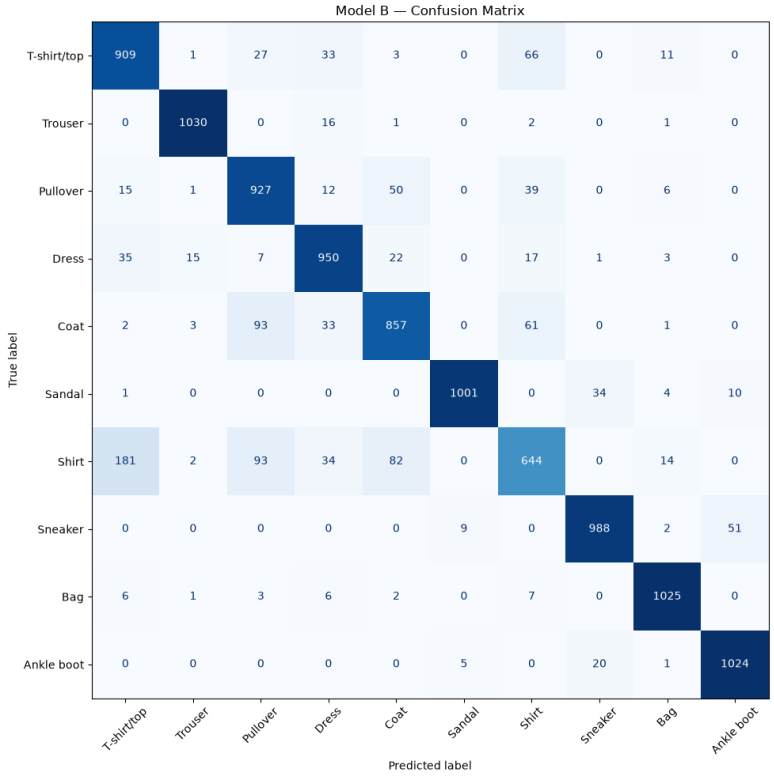


Figure 6: Model B test-set confusion matrix.

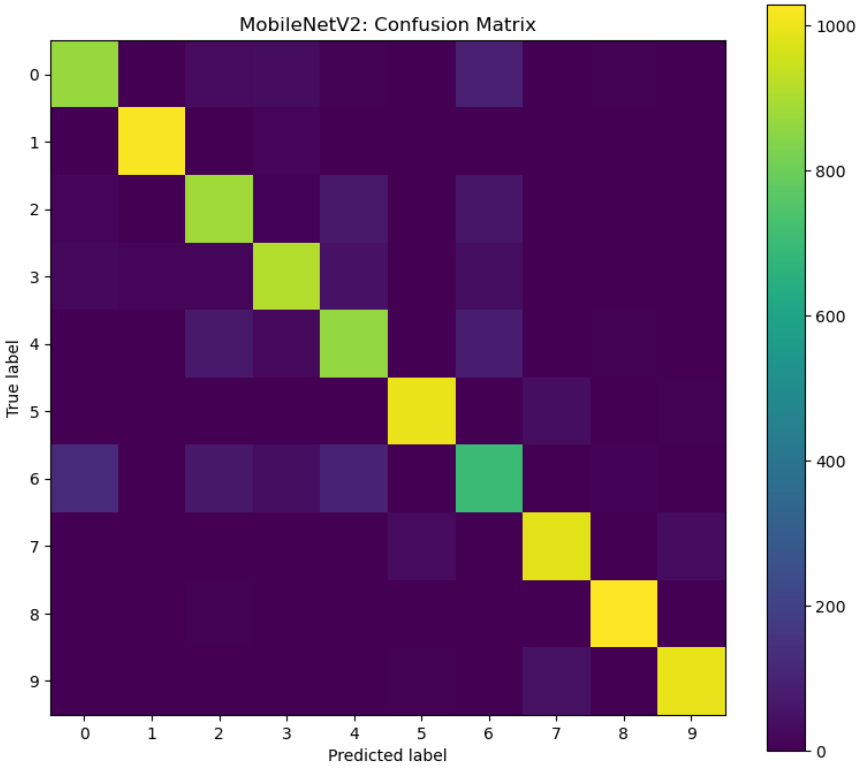


Figure 7: MobileNetV2 test-set confusion matrix.

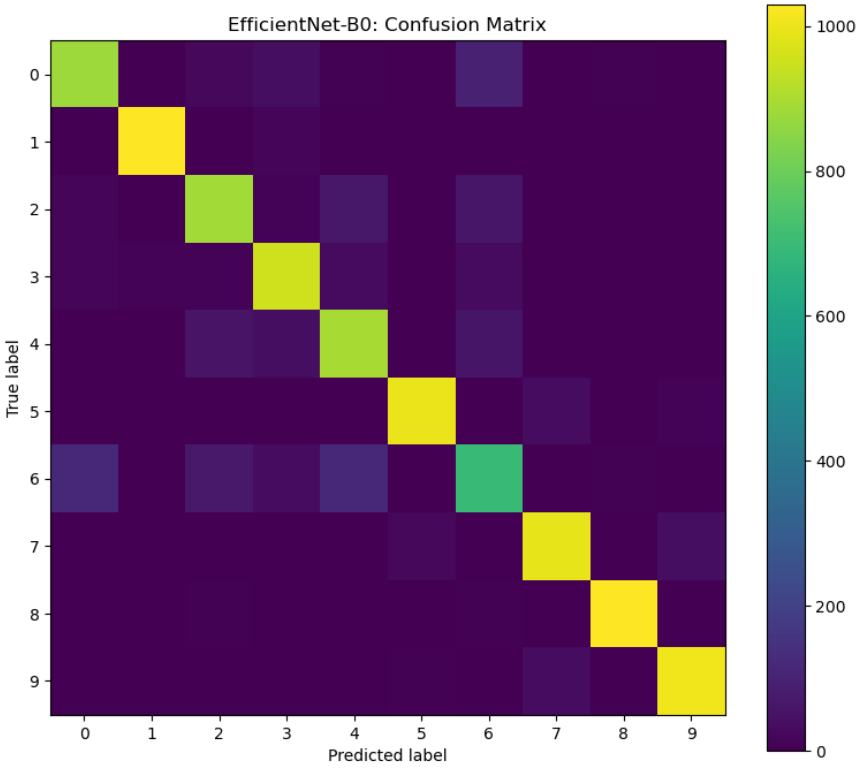


Figure 8: EfficientNetB0 test-set confusion matrix.

5.5 Pretrained model learning curves

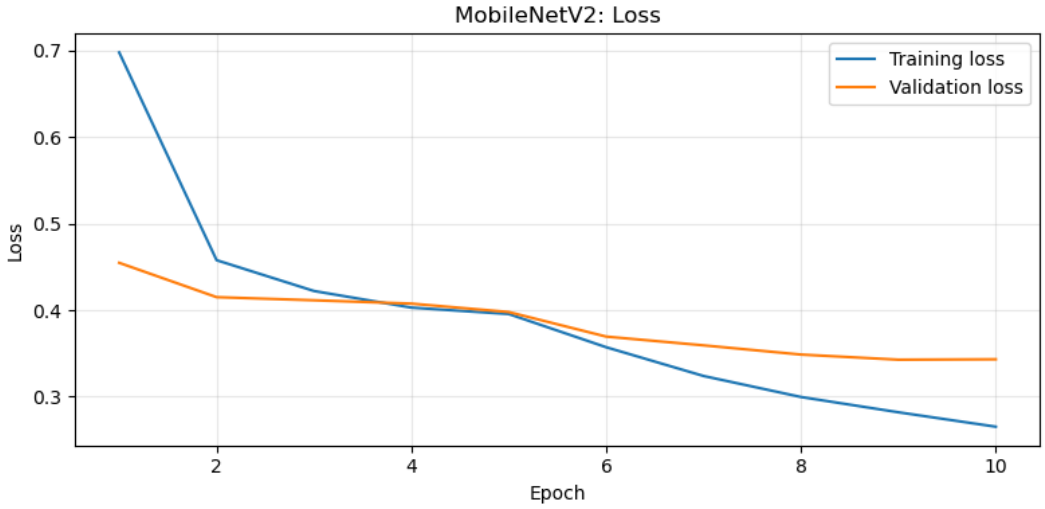


Figure 9: MobileNetV2 training and validation loss curves.

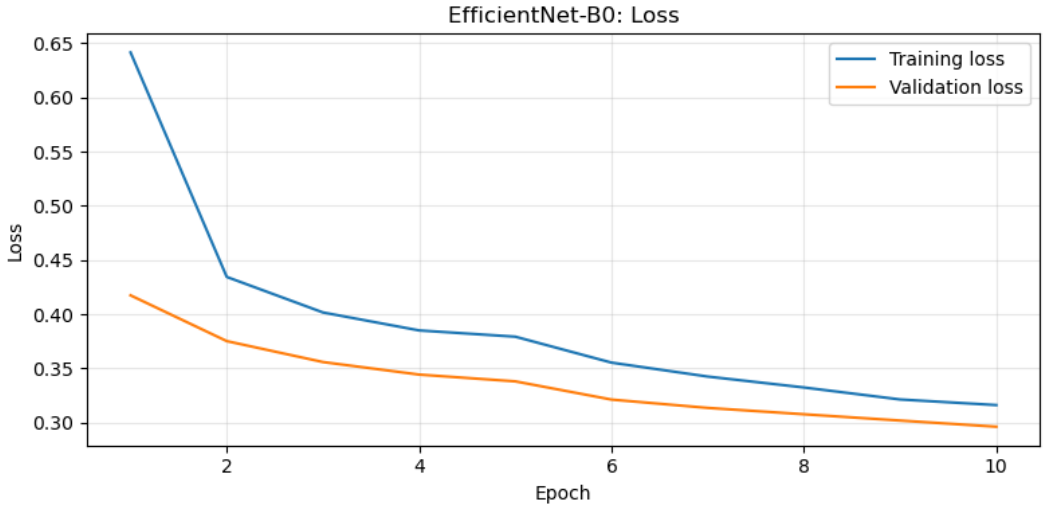


Figure 10: EfficientNetB0 training and validation loss curves.

6 Resource Efficiency and Model Comparison

6.1 Custom model comparison

Table 8: Required comparison of the two custom CNNs

Metric	Model A	Model B
Total parameters	60,874	4,738
Estimated/saved model size (KB)	757.90	18.51
Average training time per epoch (s)	17.17	7.52
Test accuracy (%)	89.58	89.10

Discuss whether depthwise separable convolutions reduced parameter count and saved model size, and how they affected accuracy and training time. Do not assume Model B is faster solely because it has fewer parameters: runtime depends on hardware, batch size and operation implementation.

6.2 Comparison with lightweight pretrained networks

Table 9: Comparison of Model B with pretrained lightweight models

Metric	Model B	MobileNetV2	EfficientNetB0
Total parameters	4,738	2,270,794	4,062,381
Trainable parameters after fine-tuning	4,354	1,207,690	1,355,578
Saved model size (MB)	0.018	8.67*	15.51*
Average training time per epoch (s)	7.52	40.38	61.16
Test accuracy (%)	89.10	88.23	89.29
Weighted precision	0.8902	0.8815	0.8921
Weighted recall	0.8910	0.8823	0.8929
Input resolution	$28 \times 28 \times 1$	$64 \times 64 \times 3$	$64 \times 64 \times 3$

6.3 Accuracy, memory and computation trade-offs

1. **Highest test accuracy:** Model A achieved the highest test accuracy of 89.58%, outperforming Model B (89.10%) by 0.48 percentage points.

2. **Model size:** Model B has only 4,738 parameters, compared with 2,270,794 for MobileNetV2 and 4,062,381 for EfficientNetB0. This makes Model B much more compact. Actual saved model sizes should be compared using the exported file sizes.

3. **Effect of pretraining:** ImageNet pretraining did not improve accuracy over Model A. MobileNetV2 achieved 88.23%, while EfficientNetB0 achieved 89.29%. However, EfficientNetB0 performed slightly better than Model B.

4. **Training time:** The model with the shortest measured training time per epoch should be identified from the experimental results. Training time alone does not determine inference speed or energy use, which depend on the hardware and implementation.

5. **Model A versus Model B:** Model B achieved nearly the same accuracy as Model A while reducing the parameter count by approximately 92.22%. This makes it a useful option when memory is limited, with only a small reduction in test accuracy.

6. **Overall comparison:** Model B offers a compact and efficient architecture, while Model A provides the highest test accuracy. Pretrained models have substantially more parameters but can benefit from previously learned features. The best choice depends on the application's accuracy and resource requirements.

The experiments show that a smaller model can achieve competitive accuracy without requiring a large pretrained architecture. For reproducibility, the final report should state the experimental hardware, software versions, and timing method used.

7 Limitations

Fashion-MNIST is a low-resolution dataset with ten classes, so results may not generalize to more complex real-world imagery. The pretrained networks receive resized three-channel inputs, whereas custom CNNs can operate directly on grayscale images; this difference should be considered when interpreting computation and input-processing overhead.

Training time depends on CPU/GPU hardware, runtime configuration, batch size, software versions and whether data loading is included. Saved model size depends on serialization format. Parameter count, file size and training time do not alone measure on-device inference latency, peak RAM use, energy consumption or multiply-accumulate operations.

8 Conclusion

This assignment compared two custom CNN architectures with two pretrained image-classification networks on Fashion-MNIST. Model A provided a conventional CNN baseline, while Model B used depthwise separable convolutions to reduce the parameter count.

Model A achieved the highest test accuracy of 89.58%, followed by EfficientNetB0 at 89.29%, Model B at 89.10%, and MobileNetV2 at 88.23%. With only 4,738 parameters, Model B achieved competitive accuracy while using far fewer parameters than the pretrained models. Overall, the results show that a carefully designed lightweight CNN can achieve comparable performance with much lower model complexity, making it a practical choice when memory and computational resources are limited.

9 Reproducibility and GitHub Repository

GitHub repository: https://github.com/Saruka03/EN3150-CNN-Classification
The repository should contain the five Jupyter notebooks and a README describing dependencies, dataset preparation, execution order and required supporting files.

Hardware used: Notebook logs report no available GPU for the Model B run; CPU model and RAM were not recorded in the exported outputs.

Software versions: TensorFlow 2.21.0 is recorded in the Model B notebook; other package versions were not consistently recorded.

10 References

1. H. Xiao, K. Rasul and R. Vollgraf, "Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms," 2017. Available: https://github.com/zalandoresearch/fashion-mnist.

2. A. G. Howard et al., "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," arXiv:1704.04861, 2017. Available: https://arxiv.org/abs/1704.04861.

3. M. Sandler et al., "MobileNetV2: Inverted Residuals and Linear Bottlenecks," CVPR, 2018. Available: https://arxiv.org/abs/1801.04381.

4. M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," ICML, 2019. Available: https://arxiv.org/abs/1905.11946.

5. TensorFlow, "Transfer learning and fine-tuning." Available: https://www.tensorflow.org/guide/keras/transfer_learning.

6. TensorFlow, "Keras Applications." Available: https://www.tensorflow.org/api_docs/python/tf/keras/applications.

7. EN3150 Pattern Recognition, "Assignment 03: Resource-Constrained CNN for Edge Image Classification," course assignment brief, University of Moratuwa, 2026.